

МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
им. Н.Э. Баумана

Факультет “Информатика и системы управления”
Кафедра “Системы обработки информации и управления”



Дисциплина “Парадигмы и конструкции языков программирования”

Отчет по лабораторной работе №1
“Основные конструкции языка Python”

Выполнил:
Студент группы ИУ5-34Б
Прощаев А.П.
Преподаватель:
Нардид А.Н.

Москва 2025

Постановка задачи

Разработать программу для решения биквадратного уравнения.

1. Программа должна быть разработана в виде консольного приложения на языке Python.
2. Программа осуществляет ввод с клавиатуры коэффициентов A , B , C , вычисляет дискриминант и действительные корни уравнения (в зависимости от дискриминанта).
3. Коэффициенты A , B , C могут быть заданы в виде параметров командной строки. Если они не заданы, то вводятся с клавиатуры в соответствии с пунктом 2.
4. Если коэффициент A , B , C введен или задан в командной строке некорректно, то необходимо проигнорировать некорректное значение и вводить коэффициент повторно пока коэффициент не будет введен корректно. Корректно заданный коэффициент – это коэффициент, значение которого может быть преобразовано без ошибок в действительное число.

Код программы

main.py

```
import sys
import math

NO_ROOTS = 0
ONE_ROOT = 1
TWO_ROOTS = 2
FOUR_ROOTS = 4

COMMAND_LINE_PARAMETERS_SHIFT = 1

COEFFICIENTS_NUMBER = 3

def GetCoefficient(index):
    coefficientString = ReadCoefficient(index)

    while True:
        try:
            coefficient = float(coefficientString)
            return coefficient
        except:
            print('Invalid coefficient')
            coefficientString = ReadCoefficientFromInputStream(index)

def ReadCoefficient(index):
    try:
```

```

        coefficientString = ReadCoefficientAsCommandLineParameter(index)
    except:
        coefficientString = ReadCoefficientFromInputStream(index)
    return coefficientString

def ReadCoefficientAsCommandLineParameter(index):
    coefficientString = sys.argv[index + COMMAND_LINE_PARAMETERS_SHIFT]
    return coefficientString

def ReadCoefficientFromInputStream(index):
    coefficientLetters = "ABC"
    print(f"Enter coefficient {coefficientLetters[index]}:")

    coefficientString = input()
    return coefficientString

def CalculateRootsOfQuadraticEquation(a, b, c):
    discriminant = CalculateDiscriminant(a, b, c)

    if discriminant < 0.0:
        return (NO_ROOTS,)
    elif discriminant == 0.0:
        rootOfQuadraticEquation = -b / (2.0*a)
        return (ONE_ROOT, rootOfQuadraticEquation)
    elif discriminant > 0.0:
        sqrtDiscriminant = math.sqrt(discriminant)
        root1 = (-b + sqrtDiscriminant) / (2.0*a)
        root2 = (-b - sqrtDiscriminant) / (2.0*a)
        return (TWO_ROOTS, root1, root2)

def CalculateDiscriminant(a, b, c):
    return b*b - 4*a*c

def CalculateRootsOfBiquadrateEquation(a, b, c):
    rootsTuple = CalculateRootsOfQuadraticEquation(a, b, c)

    match rootsTuple:
        case (NO_ROOTS,):
            return rootsTuple
        case (ONE_ROOT, root):
            return CalculateBiquadraticRootsForQuadraticRoot(root)
        case (TWO_ROOTS, root1, root2):
            return CalculateBiquadraticRootsIfTwoQuadraticRoots(root1, root2)

def CalculateBiquadraticRootsIfTwoQuadraticRoots(root1, root2):
    rootsTuple1 = CalculateBiquadraticRootsForQuadraticRoot(root1)
    rootsTuple2 = CalculateBiquadraticRootsForQuadraticRoot(root2)

    rootsNumber = rootsTuple1[0] + rootsTuple2[0]

    if (rootsNumber == NO_ROOTS):
        return (NO_ROOTS,)
    elif (rootsNumber == TWO_ROOTS):

```

```

        return rootsTuple1 if (rootsTuple1[0] == TWO_ROOTS) else rootsTuple2
    elif (rootsNumber == FOUR_ROOTS):
        return (FOUR_ROOTS, rootsTuple1[1], rootsTuple1[2], rootsTuple1[1],
rootsTuple2[2])

def CalculateBiquadraticRootsForQuadraticRoot(root):
    if root < 0:
        return (NO_ROOTS,)
    elif root == 0:
        return (TWO_ROOTS, 0, 0)
    elif root > 0:
        return (TWO_ROOTS, -math.sqrt(root), math.sqrt(root))

def PrintRoots(rootsTuple):
    match rootsTuple:
        case (NO_ROOTS,):
            print('No roots')
        case (ONE_ROOT, root):
            print(f'One root: {root}')
        case (TWO_ROOTS, root1, root2):
            print(f'Two roots: {root1} и {root2}')
        case (FOUR_ROOTS, root1, root2, root3, root4):
            print(f"Four roots: {root1}, {root2}, {root3} and {root4}")

def main():
    coefficientList = []

    for i in range(COEFFICIENTS_NUMBER):
        coefficientList.append(GetCoefficient(i))

    roots = CalculateRootsOfBiquadrateEquation(*coefficientList)
    PrintRoots(roots)

if __name__ == "__main__":
    main()

```

Результат работы программы

```

● proshchaevalexey@MacBook-Air-Aleksej-2 lab_1 % python3 main.py 1 0 -16
Two roots: -2.0 и 2.0
● proshchaevalexey@MacBook-Air-Aleksej-2 lab_1 % python3 main.py 1 -5
Enter coefficient C:
6
Four roots: -1.7320508075688772, 1.7320508075688772, -1.7320508075688772 and 1.4142135623730951
● proshchaevalexey@MacBook-Air-Aleksej-2 lab_1 % python3 main.py
Enter coefficient A:
something
Invalid coefficient
Enter coefficient A:
2
Enter coefficient B:
0
Enter coefficient C:
20
No roots
○ proshchaevalexey@MacBook-Air-Aleksej-2 lab_1 %

```